



PYTHON

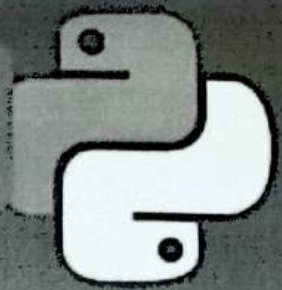
~ Arijit Das



```
import random
print(random.choice([
    "You can do it!",
    "Keep going!",
    "Believe in yourself!",
    "Stay positive!",
    "One step at a time."
]))
```

#Quote #Programming #Selfcare

PYTHON MINDMAP



BASICS

- Syntax
- Variables
- Data Types
- Control Structures

OBJECT-ORIENTED PROGRAMMING (OOP)

- Classes & Objects
- Inheritance
- Encapsulation
- Polymorphism

LIBRARIES AND FRAMEWORKS

- NumPy
- Pandas
- Matplotlib
- TensorFlow
- Flask

DATA SCIENCE

- Pandas
- NumPy
- Matplotlib
- Scikit-learn
- TensorFlow

FUNCTIONS

- Built-in Functions
- Defining
- Functions
- Recursion

FILE HANDLING

- Reading with CSV
- Working with JSON

CONCURRENCY

- Threading
- Multiprocessing
- Asyncio

TOOLS AND IDEs

- PyCharm
- Jupyter Notebook
- Visual Studio Code
- Anaconda

DATA STRUCTURES

- Lists
- Tuples
- Dictionaries
- Sets

MODULES AND PACKAGES

- Importing Modules
- Creating Packages
- Using Packages

WEB DEVELOPMENT

- Django
- Flask

TOOLS AND IDEs

- PyCharm
- Jupyter Notebook
- Visual Studio Code
- Anaconda
- Plp

|| PYTHON ||

▣ What is Python?

⇒ Python is a beginner-friendly, high-level, general purpose and Object oriented programming language.

▣ Why should we learn Python?

⇒

- Easy to use & learn.
- Expressive language.
- Interpreted language.
- Object-oriented language.
- Open-source language.
- Extensible.
- Dynamic memory allocation.
- GUI Programming support.

▣ Where is Python used?

⇒

- Data Science.
- Data Mining
- Desktop Application
- Mobile Application
- Web Application
- Machine Learning
- Artificial Intelligence
- Speech Recognition

Python Popular Frameworks & Libraries.

- ① Web Development: Django, Flask, Pyramid, CherryPy.
- ② GUIs based Applications: Tk, PyGTK, PyQT, PyJS.
- ③ Machine learning: TensorFlow, PyTorch, Scikit-learn.
- ④ Mathematics: NumPy, Pandas.

▣ First Program:

Print("Hello World")

• As a integer

Print(2)

• As a string

Print('2')

output

Hello World

output

2

output

2

▣ Addition, Multiplication, division, subtraction:

Print(2+3)

Print(2*3)

Print(2/3)

Print(2-3)

output

5

output

6

output

0.6

output

-1

▲ NOTE

We can print it only in one line by print() function.

▣ Comments in Python:

Symbol: #

① # Printing helloworld
print("Hello world")

② # Addition of two nums
print(2+3)

③ # Multiplication of two nums
print(2*3)

▲ NOTE:

Comments are used to explain codes, make it readable, and help other developers to understand the code.

① Single-line Comment

This is called single line comment.

② Multiline Comment

''' This is called
multiline comment '''

```
y = 10  
print(y)
```

▲ NOTE :

These are technically multiline strings, but they can act like comments if not assigned to a variable or used in a code.

CHARACTER SET in Python

1.

Letters :-

- Lower case : $a \rightarrow z$
- Upper case : $A \rightarrow Z$

2.

Digits :-

- $0 \rightarrow 9$

3.

Special characters :-

@, #, \$, %, &, *, (,), {, }, [,], :, ;, . etc

4.

White space characters & / Escape Sequence characters :-

i

Space (' ')

ii

Tab (\t)

iii

Newline (\n)

iv

Carriage Return (\r)

v

Form Feed (\f)

vi

Vertical Tab (\v)

1

space (' ')

print("Hello World")

Output :-

Hello World

ii

Tab (\t)

print("Arijit \t Das")

output :-

Arijit Das

iii

Newline (\n)

print("I am a \n Student")

output :-

I am a
Student

iv

Carriage return (\r)

Same as New line (\n)

Variables

A variable in python [in any programming language] is a name given by the user which are located in memory. Variable helps to store data types. [Integer, string, float, bool] and other data structures [list, dictionary, tuple]. Basically it stores the data.

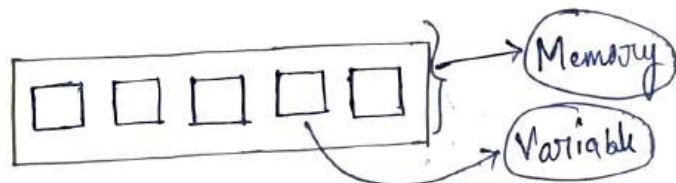
* Example :-

name = "Ajijit" → Here name is the variable which stores the string.

age = 23 → Here age is the variable which stores the integer.

price = 25.99 → Here price is the variable which stores the float.

* The variables are stored in a MEMORY.



List = [1, 2, 3, 4, 5]

tuple = (1, 2, 3)

dict = { 1: "Ajijit", 2: "address" }

Set = { 1, 2, 3, 4 }

Here, the variables are list, tuple, dict, set

* Example :-

x = 10 # integer

y = 20.0 # float

name = "Ajijit" # string

is_active = True # Bool

* Variable naming Rule :-

1. Variable names can be a combination of upper case or lower case letters, digits or an underscore.

myName = "Ajijit"

myDigit1 = 12

Variable_name = "Sourav"

2. Variable name cannot start with a number.

1 Variable = X [Wrong]

Variable 1 = ✓ [Correct]

(ii) We cannot use special characters to make a variable name.
like $\rightarrow$!, #, @, %, \$

(iv) Variables can be any length. [len()]

(v) Dynamic Typing

Python dynamic typing is the variable data can be changed at any time.

* Ex:-

```
x = 1
print(x)
x = 2
print(2)
```

output:-

1
2

* Ex:-

```
x = 1
x = 2
print(x)
```

output:-

2

* Ex:-

```
x = 1
x = 2.0
print(x)
```

output:-

2.0

* Ex:-

```
x = 12
x = "name"
print(x)
```

output

name

* Ex:-

```
x = "name"
x = "Amit"
print(x)
```

output

Amit

* Ex:-

```
x = 1
x = 22.0
print(x)
```

output:-

22.0

(i) Multiple Assignments

```
a, b, c = 1, 2, 3
print(a, b, c)
```

output:-

1, 2, 3

(ii) $a = b = c = 1, 2, 3$
`print(a, b, c)`

output:- 1 2 3

DATA TYPES

There are 5 types of data types.

- Integer
- String
- Float
- Bool
- None

④ Integer Data types

The integer data types in python represents the whole numbers (Positive, Negative, Zero) without any 'decimal or fractional part'
[Numbers starting from Zero]

(i) Declaring in Python

```
x = 10    # Positive integer
y = -5    # Negative integer
z = 0     # Zero
```

ii) checking type

```
* age = 20
print(type(age))
```

output:

<class 'int'>

```
* x = 55
print(type(x))
```

output:

<class 'int'>

```
* number_1 = 30
print(type(number_1))
```

Output:

<class 'int'>

```
* y = -5
print(print(y))
print(type(y))
```

output:

<class 'int'>

② String Data type
String in Python is a sequence of characters enclosed in single ('), double ("), or triple quotes (''' or ''').

(i) Declaring in Python :-

```
String-Var1 = "Hello" # "double quote"
```

```
String-Var2 = 'Hello' # 'single quote'
```

```
String-Var3 = '''Hello''' # 'triple quote'
```

```
print (type (String-Var1))
```

```
print (type (String-Var2))
```

```
print (type (String-Var3))
```

output:-

```
<class 'str'>
```

```
<class 'str'>
```

```
<class 'str'>
```

(ii) Multiline string written in triple quotation.

```
String-Var = '''Arijit
```

```
das'''
```

```
print (String-Var)
```

output:-

```
Arijit
```

```
das
```

③ Float Data Type:

A float data type in python represents decimal or real numbers, allowing fractional values.

(i) Declaring a float:

$x = 2.14$ # A normal float

$y = -2.12$ # A negative float

$z = 1.0$ # A float with a whole [0]

`print (type (x))`

`print (type (y))`

`print (type (z))`

Output:

<class 'float'>

<class 'float'>

<class 'float'>

(ii) Scientific Notation : [Exponential]

$x = 3e4$

`print(x)`
`print(type(x))`

output:

30,000

<class 'float'>

A float can also be written in scientific notation using 'e' or 'E', where 'e' represents $\boxed{\times 10^{\wedge}}$

Dry Run:

$x = 3e4$ # 3×10^4

$= 3 \times 10 \times 10 \times 10 \times 10$

$= 30,000$

S1 = ''' Hello

World'''

print(S1)

output

Hello

World

S2 = ''' Hello world'''

print(S2)

output

Hello world

Concatenation (+)

S1 = "Hellow"

S2 = " World"

result = S1 + " " + S2

print(result)

output:

Hellow World

Repetition

string = "Hi "

print(string * 3)

output:

Hi Hi Hi

string = "Hi "

print(string * 3)

output:

Hi

Hi

Hi

Strings

Definition :-

String is a data type that stores a sequence of characters.

Indexing :-

A r i j i t → index
0 1 2 3 4 5

Index are helps us to access characters.

Index → Position of each letter (upper/lower).

It always starts from 0.

Syntax :-

stor []

Example :-

"A r i j i t - D a s"
0 1 2 3 4 5 6 7 8 9

→ stor [0] = 'A'

→ stor [8] = 'a'

stor = "A r i j i t"

ch = stor [0]

print (ch)

output :-

A

stor = "A r i j i t"

ch = stor [1]

print (ch)

output :-

r

stor = "A r i j i t"

ch = stor [2]

print (ch)

output :-

i

③ String slicing :-
It helps to accessing parts of a strings.

* Syntax :-

str [index : index]
(starting) : (ending)

* Programme :-

```
str = "Arijit"
print(str[0:6])
```

print only 'ari and i'

```
str = "Arijit"
print(str[0:5])
```

print only 'iji'

```
str = "Arijit"
print(str[2:5])
```

print only 'Ari'

```
str = "Arijit"
print(str[0:3])
```

print only 'jit'

```
str = "Arijit"
print(str[3:6])
```

print only 'Arijit'

```
str = "Arijit"
print(str[:])
```

④ output :-
Arijit

④ output :-
Ariji

④ output :-
iji

④ output :-
Ari

④ output :-
jit

Another method

```
str = "Arijit"
print(str[0:7])
```

output
Arijit

④ Basically it takes the first index which is written for example ↴

str[0:6] A
↓
first index

and ignores the last written index and prints the previous index. for example ↴

str[0:5] i
↓
last index

① Negative string slicing:-

" s t o i n g "

-6	-5	-4	-3	-2	-1
----	----	----	----	----	----

→ count →

* Syntax:-

[starting index : - ending index]

* Program:-

Print only 'i n g'

```
str = " s t o i n g "
```

```
print (str[-3 :])
```

* output
ing

Print only 's t o i'

```
str = " s t o i n g "
```

```
print (str[-6 :-3])
```

* output
s t o i

Print only 'o i n'

```
str = " s t o i n g "
```

```
print (str[-4 :-1])
```

* output
o i n

Print only 's t o i'

```
str = " s t o i n g "
```

```
print (str[-6 :-2])
```

* output
s t o i

Print only 't o i'

```
str = " s t o i n g "
```

```
print (str[-5 :-2])
```

* output
t o i

String Functions

- ① str.capitalize() ④ str.lower()
- ② str.endswith()
- ③ str.find()
- ④ str.count()
- ⑤ str.replace()
- ⑥ str.upper()

① str.capitalize()

This method is used to write the first letter in capital.

* Program:-

```
str = "i am Arijit"  
print(str.capitalize())
```

Output:-

I am Arijit

② str.endswith()

This method is used to returns True if the string ends with the same substr given as a argument in the method. Or if the substr is different then it will give false as a output.

* Program:-

```
str = "Arijit"  
print(str.endswith("it"))
```

Output:-

True

* Program:-

```
str = "Arijit"  
print(str.endswith("ln"))
```

Output:-

false

③ str.find(" ")

This method is used to find the index no. by given the specific substr in the bracket.

⊗ Program:-

output:-

```
str = "Arjit"  
print(str.find("t"))  
print(str.find("j"))  
print(str.find("i"))
```

5

3

1

④ str.count(" ")

output:-

```
str = "I am Arjit"  
print(str.count("i"))  
print(str.count(" "))  
print(str.count("m"))
```

2

2

1

The space " " is a character in python. that's why it returns 2 as a output. Because there are two space characters.

This method helps us to count the no. of characters which are present as a string.

⑤ str.replace(old, new)

This method is used to replace the one character to another character.

⊗ Program:-

```
str = "Arjit is a student"  
print(str.replace("student", "boy"))
```

output:-

Arjit is a boy

⑥ str.upper()

This method is used to print the all characters in uppercase.

⊗ Program:-

```
str = "arjit"  
print(str.upper())
```

output:-

ARJIT

⑦

str.lower()

This method is used to print the all characters in lower.

Program:-

str = "ARJIT"

print(str.lower())

Output:-

arjit

⑧

WAP to print a name from user & print it's length.

str = raw-input("Enter your name :")

print(len(str))

print("length of your name:", len(str))

Output:-

Enter your name: Arjit

6

length of your name: 6

④

Bool data type :

The bool data type represents boolean values like, True and False. It commonly used in conditional statements and logical operations.

(i)

Declaring Boolean Variables
 $x = \text{True}$
 $y = \text{False}$
`print (type (x))`
`print (type (y))`
output:-

<class 'bool'>

<class 'bool'>

⊗

True is treated as = 1 | True = 1

⊗

False is treated as = 0 | False = 0

(ii)

Boolean as numbers:

True = 1

False = 0

`print (True + 1)`
`print (False + 1)`
output:-

2

1

⊗ Dry Run
 $\rightarrow (1) + 1 = 2$

(True)

 $\rightarrow (0) + 1 = 1$

(False)

print(True * 2)

print(False * 2)

print(True / 2)

print(False / 2)

output:

2

0

0.5

0.0

(iv)

Comparison operators:

$>$, $<$, $==$, $!=$, $<=$, $>=$

print (5 > 2) # True

print (5 < 2) # False

print (5 == 2) # False

print (5 >= 2) # True

print (5 <= 2) # False

Types of operators

* There are four types of operators.

- i) Arithmetic operator $[+, -, *, \%, /, **]$
- ii) Relational / comparison operator $[==, >, <, >=, <=, !=]$
- iii) Assignment operator $[=, +=, -=, /=, \%=, **=]$
- iv) Logical operator $[not, and, or]$

i) ARITHMETIC OPERATOR :-

- a) $a = 5 + 3$ # output : 8 [Addition (+)]
- b) $a = 10 - 4$ # output : 6 [Subtraction (-)]
- c) $a = 6 * 7$ # output : 42 [multiplication (*)]
- d) $a = 10 / 2$ # output : 5 [Division (/)]
- e) $a = 10 // 3$ # output : 3 [Floor division (//)]
- f) $a = 10 \% 3$ # output : 1 [Modulus (%)]
- g) $a = 2 ** 3$ # output : 8 (2^3) [Exponential (**)]

important ↓

- i) Floor division (//) → Divides and rounds down the nearest integer.
- ii) Modulus (%) → Returns the remainder of a division.
- iii) Exponential (**) → Raises a number to the power of another.

ii) Relational Operator: $[==, >, <, >=, <=, !=]$

- $==$ → Equal to
- $>$ → Greater than
- $<$ → less than
- $>=$ → Greater than equal to
- $<=$ → less than equal to
- $!=$ → Not equal to

Assignment operators:

$=$ $\rightarrow$ used for store value of any data type

$+=$ $\rightarrow$ ex: $\rightarrow$ $a = 10$ | output
 $a += 10$ | 20
 $\text{print}(a)$

$-=$ $\rightarrow$ ex: $\rightarrow$ $a = 10$ | output
 $a -= 10$ | 0
 $\text{print}(a)$

$*=$ $\rightarrow$ ex: $\rightarrow$ $a = 10$ | output
 $a *= 10$ | 100
 $\text{print}(a)$

$/=$ $\rightarrow$ ex: $\rightarrow$ $a = 10$ | output
 $a /= 2$ | 5.0
 $\text{print}(a)$

Logical operators:

• **and** Returns always boolean value [T/F]
 $a = 10$ | It returns True if two conditions are true.
 $b = 20$
 $\text{print}(a < b \text{ and } b > a)$ | **## True**

• **or** = It returns True if only one condition is true between two
 $a = 10$
 $b = 20$
 $\text{print}(a < b \text{ or } b < a)$ | **## True**

• **NOT**
 ~~$a = [1, 2, 3]$~~ $x = 5$
 ~~$a = b$~~ if $\text{not } x > 10$:
 ~~$b = [1, 2, 3]$~~ $\text{print}(\text{"True"})$ | **## True**
 print

Taking user as Input:

We can usually take input as a integer, float, string and bool. But the compiler take the input usually as a string by default. We can take input by waiting only → input() function.

⊛ Here's why

First prog

```
① a = input("Enter first num:")  
b = input("Enter second num:")  
Sum = a + b  
print(f"Sum: ", Sum)
```

output:

Enter first num: 14

Enter second num: 12

Sum: 1412

Reason:

Why this happen??

This happens when we does not specifically write "int, float, str". It returns a string character (concatination) (1412)

```
① a = int(input("Enter first num:"))  
b = int(input("Enter second num:"))  
Sum = a + b  
print(f"Sum: ", Sum)
```

output:

Enter first num: 14

Enter second num: 12

Sum: 26

Reason II

Now we specifically write 'int', so it returns an integer number after sum

Taking integer as a input

First Prog:

```
① a = input("Enter first num:")  
b = input("Enter second num:")  
print(type(a))  
print(type(b))
```

Output:

<class 'str'>

<class 'str'>

```
① a = int(input("Enter first num:"))  
b = int(input("Enter first num:"))  
print(type(a))  
print(type(b))
```

Output:

<class 'int'>

<class 'int'>

Taking string as a input:-

```
name = str(input("Enter your name:"))  
print(f"Name: {name}!")
```

Output:

~~name:~~ Enter your name: Anirudh
name: Anirudh!

Taking float as a input:

```
a = float(input("Enter your height:"))  
print(f"your height is: {a}!")
```

output:

Enter your height : 6.5
Your height is : 6.5!

Taking bool as a input:

- we "yes"/"no"
- `strip().lower()`

.strip()

.strip() function used for removing the leading space between the characters

eg →

* Program :-

① WAP a program to write "yes" as "True" and "No" as "False".

```
name = str(input("Enter yes for \"True\" and No for \"False\""))  
• strip().lower() == "yes"
```

```
print(name)
```

output:

Enter yes for "True" and No for "false": yes

True

② N/A to print 1 for True and 0 for false.

```
num = int(input("Enter 1 for True and 0 for False:"))  
    .strip().lower() == '1'  
  
print(num)
```

output:

Enter 1 for "True" and 0 for "False": 1

True

How it works:-

- input() takes the user input
- .strip().lower() removes spaces and turns the letters into lower case.
- == "1" comparison returns True if the input is "yes", otherwise False.

Type Casting and Type Conversion

There are two type of data type conversion in python.

- ① → Implicit Type casting [Type conversion]
- ② → Explicit Type casting

① Implicit Type casting ① int → float

That type of type casting are done by the compiler.

eg →

```
int = 12      a = 12
float = 13.5  b = 13.5
x            x = a + b
               print(x)
```

output:
25.5

That is called implicit type casting. Here a is a integer variable and b is a float variable. After adding (int + float), the compiler returns a float value in return.

Basically,

Implicit type casting are done by the compiler.

 `print(type(a)) # <class 'int'>`

`print(type(b)) # <class 'float'>`

`print(type(x)) # <class 'float'>`

② int → complex

a = 13

b = 3 + 2j

x = a + b

print(x)

`print(type(a)) # 'int'`

`print(type(b)) # 'complex'`

`print(type(x)) # 'complex'`

output:

16 + 2j

(iii)

Boolean (bool) $\rightarrow$ Integer (int)

Python considers bool as true and false.

True = 1

False = 0

eg $\Rightarrow$

a = True

b = 5

x = a + b

print(x)

```
print(type(a)) # <class 'bool'>
print(type(b)) # <class 'int'>
print(type(x)) # <class 'int'>
```

output:

6

(iv)

Characters (str) is not implicitly converted to numbers.

a = "100"

b = a + 90

print(b)

output:

#Type Error: cannot add str + int.

(B) Explicit Type Casting:

Explicit type casting in python is also called "manual type casting".

i)

float to int

```
a = 10.0  
print(int(a))  
print(type(a))
```

output:

```
10  
<class 'int'>
```

Here, 'a' is a float variable, but we can convert it into an integer manually.

→ At first the a variable type was <class 'float'>
→ After the conversion it is <class 'int'>

ii)

```
a = 10
```

int to float

```
print(float(a))  
print(type(float(a)))
```

output

```
10.0  
<class 'float'>
```

iii)

int to bool

```
a = 1  
print(bool(a))  
print(type(bool(a)))
```

output:

```
True  
<class 'bool'>
```

```
a = 0
```

```
print(bool(a))  
print(type(bool(a)))
```

output:

```
False  
<class 'bool'>
```

iv) Bool to integer

```
a = True
print(int(a))
print(type(int(a)))
```

output

1

<class 'int'>

```
a = False
print(int(a))
print(type(int(a)))
```

output

0

<class 'int'>

v) float to bool

```
a = 1.0
print(bool(a))
print(type(bool(a)))
```

output

True

<class 'bool'>

```
b = 0.0
print(bool(b))
print(type(bool(b)))
```

output

False

<class 'bool'>

vi) bool to float

```
a = True
print(float(a))
```

Output

1.0

```
a = False
print(float(a))
```

output

0.0

vii) int to string

```
a = 12
print(str(a))
print(type(str(a)))
```

output

12

<class 'str'>

```
xiii) String to int
a = "12"
print(int(a))
print(type(int(a)))
```

12

<class 'str'>

<class 'int'>

xiv

str to bool float

a = "1"

print(float(a))

output

1.0

xv

str to bool

a = "1"

print(bool(a))

output

True

xv

float to str

b = 1.0

print(str(b))

output

1.0

xvi

bool to str

a = True

print(str(a))

print(type(str(a)))

output

True

<class 'str'>

Conditional statements

① If statement:

if condition:
 statement } → Syntax

(i) WAP to check the number is greater than 10 or not.

```
num = int(input("Enter a number: "))
```

```
if num > 10:
```

```
    print("number is greater than 10")
```

```
print("Program ended")
```

Output:

Enter a number: 15

number is greater than 10

Program ended

(ii) Exam passing condition using if statement.

```
marks = int(input("Enter your marks: "))
```

```
if (marks > 21):
```

```
    print("Pass")
```

Output:

Enter your marks: 25

Pass

③ Check the number is odd or even.

```
num = 10
```

```
if (num % 2 == 0)  
    print("Even")
```

output:

Even

④ WAP to check a user defined string length is greater than 10 or not.

```
str = input("Enter a string: ")
```

```
x = len(str)
```

```
print("length:", x)
```

```
if x > 10:
```

```
    print("string length is greater than 10.")
```

output:

Enter a string: Anirudh Das -

length: 11

string length is greater than 10.

Conditionals Statements :

There are three (3) types of conditional statements in python.

- (1) → If
- (2) → If ~~else~~ else
- (3) → if - elif - else
- (4) → Nested if

(2) IF-Else

- (1) WAP a program if your age greater than 18 then you can drive but if your age is not greater than 18 then print you can't.

```
age = int(input("Enter your age:"))
```

```
if (age > 18):  
    print("You can drive")
```

```
else:  
    print("You cannot drive")
```

If-else:- [SYNTAX]

```
if (condition):  
    statement  
else:  
    statement
```

Output:-

Enter your age : 19.
you can drive

- (2) WAP a program if it rain today then I will not go to School or I will.

```
rain_today = str(input("Enter (Yes/No) if it rain today:")).strip().lower()
```

```
if rain_today == "Yes":  
    print("I will not go to School")
```

```
else:  
    print("I will go to School")
```


Output:-

Enter(Yes/No) if it rain today : Yes

I will not go to School.

⊛ strip()

We use strip() to remove the leading or trailing spaces between the character.

⊛ Programme:-

```
myName = " Arijit "  
print(myName.strip())
```

output
Arijit

⊛ lower()

We use the lower() to print the characters in lower.

⊛ Programme:-

```
myName = "ARIJIT"  
print(myName.lower())
```

output
arijit

⊛ strip().lower()

We use this together to make the remaining ~~value~~ conditional statement come.

if rain-today == "yes"

By using strip().lower(),

if the string "yes" written as " Yes"
or "yes" or "yes", it will remain
same as "yes".

③ WAP to check the no. odd or even.

```
age = int(input("Enter a number as age :"))
```

```
if age % 2 == 0 :  
    print("Even")
```

```
else :
```

```
    print("odd")
```

Output:

Enter a number as age : 18

Even

④ WAP if your marks greater than equals to 21 then print "PASS" else print "FAIL".

```
marks = int(input("Enter your marks :"))
```

```
if marks >= 21 :  
    print("PASS")
```

```
else :
```

```
    print("FAIL")
```

output:-

Enter your marks : 70

70 PASS

If-elif-else:

Syntax:

```
if (condition):  
    statement  
elif (condition):  
    statement*  
else:  
    statement
```

① Check whether a number is positive, negative or zero.

```
num = int(input("Enter a num:"))  
  
if (num > 0)  
    print("Positive")  
elif (num < 0)  
    print("Negative")  
else:  
    print("Zero")
```

Output:

Enter a num: 18

Positive

② WAP to print the marks of your exam.

marks ≥ 90 , marks ≤ 100 , grade 'A'

marks ≥ 80 , marks ≤ 89 , grade 'B'

marks ≥ 70 , marks ≤ 79 , grade 'C'

else, print("D")

```
marks = int(input("Enter your marks: "))
```

```
if marks >= 90 and marks <= 100:  
    print("A")
```

```
elif marks >= 80 and marks <= 89:  
    print("B")
```

```
elif marks >= 70 and marks <= 79:  
    print("C")
```

```
else:  
    print("D")
```

Output:

Enter your marks: 75

C

WAP to find the greatest of 3 numbers entered by the user.

```
a = int(input("Enter first number: "))
```

```
b = int(input("Enter second number: "))
```

```
c = int(input("Enter third number: "))
```

```
if a > b and a > c:  
    print("greater number is:", a)
```

```
elif b > a and b > c:  
    print("greater number is: ", b)
```

```
else:  
    print("greater number is:", c)
```


Output:

Enter first number: 12

Enter Second number: 13

Enter third number: 95

Greater number is: 95

Q4) WAP to check the letter is A or not.

```
letter = "A"
```

```
if letter == "B"  
    print("B")
```

```
elif letter == "e"  
    print("e")
```

```
elif letter == "A"  
    print("letter is A")
```

```
else:
```

```
    print("letter is not A,B,e")
```

output:

letter is A.

⑥ WAP to check a letter is vowel or not.

```
letter = str(input("Enter a letter : ")).strip().lower()

if letter == 'a':
    print("This is a vowel")

elif letter == 'e':
    print("This is a vowel")

elif letter == 'i':
    print("This is a vowel")

elif letter == 'o':
    print("This is a vowel")

elif letter == 'u':
    print("This is a vowel")

else:
    print("This is not a vowel")
```

output:

Enter a letter: a

This is a vowel

DICTIONARY and DICTIONARY METHODS :-

Definition :

- Dictionary is a method in python to store the data in ~~immutable~~ mutable way.
- This is ~~immutable~~ (changeable).

Syntax

```
Variable-name = {  
    Key : Value,  
}
```

Programme - 1 :

```
Key : Value  
diet = {  
    int : 18,  
    float : 18.0,  
    str : "Arijit",  
    list : [18, 9, 10],  
    Tup : (1, 2, 3),  
}  
print(diet)
```

⊛ This is a set up of Key: Value pairs.

⊛ It can store string, integer, float, list, tuple data type variable as Key: Value

⊛ Mutable
(changeable)

⊛ List and Tuple are not data type

Here the first three data types and the list, tuple methods can be store in a single dictionary.

WAP in python to show all data types as: Key: Value

```
my-diet = {  
    "Integer" : 12,  
    "float-var" : 12.0,  
    "string" : "Arijit Das",  
    "list-var" : [12, 0, 9],  
    "Tup" : (1, 2, 4),  
}  
print(my-diet)
```

WAP in python to show your BIO-DATA.
print ("BIO-DATA")

diet = {

"Name": "Arjun Das",

"Age": 18,

"Address": "Dum Dum",

"Father's Name": "Deepankar Das",

"Mother's Name": "Rupali Das",

"Education": "Student",

}

print (diet)

WAP in python to show your mark sheet.

print ("Mark sheet")

diet = {

"Subject-1": 90,

"Subject-2": 90,

"Subject-3": 100,

"Subject-4": 80,

"Subject-5": 90,

}

print (diet)

* You can use another dictionary variable name also.

* It should be written as the topic as given in problem.

* Dictionary Key name should be different or it will occur a error.

* All elements in the dictionary are ~~immutable~~ / changeable.

* ~~Immutable~~ means ~~not~~ changeable.

Built in functions in Python Dictionary.

① len() → Returns the numbers of Key:Value pairs.

program

②

```
dict = {  
    1: "Arijit",  
    2: 18,  
    3: [1, 2, 3],  
    4: (12, 13, 14),  
}
```

print(len(dict))

③ → Here the no. of Key-Values are 4 So the length of dictionary is 4.

output:

4

② sorted() →

- ① For integer Key, it sort the keys numberwise.
- ② For string Key, it sort the keys alphabetically.
- * It only sort the keys not the values.

* Integer:

```
dict = {  
    1: "Arijit",  
    2: 18,  
    3: [1, 2, 3],  
    5: [4, 5, 6],  
    4: (1, 2, 3),  
}
```

print(sorted(dict))

output:

[1, 2, 3, 4, 5]

* String:-

```
dict = {  
    "name": "Arijit",  
    "Age": 18,  
    "Marks": [95, 96, 97],  
    "course": "BEA",  
}
```

print(sorted(dict))

output:

['Age', 'Marks', 'course', 'name']

③ In that case 'course' Key should be in the 2nd but the system give importance to the upper case letter first, so the 'M' is in the 2nd position after 'A'. [M = Marks, A = Age]

* dict = {

"name": "Arijit",

"age": 18

"course": "BEA",

"marks": [12, 13, 14],
}

~~print(dict)~~

print(sorted(dict))

* output

['age', 'course', 'marks', 'name']

* dict = {

"name": "Arijit",

"address": "Dum Dum",

"course": "BEA",

"subjects": "Python",

"age": 18,

}

print(sorted(dict))

output

['address', 'age', 'course', 'name', 'subjects']

DICCTIONARY METHODS()

④ clear() → Removes all the element from the dict.

i> dict = {
 1: "name",
 2: "Age",
 3: "value",
 4: "Add",
}

dict.clear()
print(dict)

output:
{}

ii> dict = {

 "name": "Arijit",

 "Age": 18,

 "address": "Dum Dum",

}

dict.clear()

print(dict)

output:

{}

② copy() → Returns a shallow copy of the dictionary.

i> dict = {
 1: "Arijit",
 2: "Alice",
 3: "John",
 4: "Adam",
}

x = dict.copy()

print(dict)

print(x)

output:

{ 1: 'Arijit', 2: 'Alice', 3: 'John', 4: 'Adam' }

ii> dict = {

 1: 18,

 2: 19,

 3: 20,

 4: 20,

}

print(dict.copy())

output:

{ 1: 18, 2: 19, 3: 20, 4: 20 }

3) .Keys() → Returns all the Keys.

i) dict = {
 1: "Arijit",
 2: "Adam",
 3: "Alice",
 4: "John",
}
print(dict.keys())

output:

dict_keys([1, 2, 3, 4])

ii) dict = {

 "name": "Arijit",
 "Age": 18,
 "Address": "Dum Dum",
}

print(dict.keys())

output:

dict_keys(['name', 'Age', 'Address'])

4) .Values() → Returns all the Values.

i) dict = {
 1: "Arijit",
 2: "Adam",
 3: "Alice",
 4: "John",
}
print(dict.values())

output:

dict_values(['Arijit', 'Adam', 'Alice', 'John'])

ii) dict = {
 "name": "Arijit",
 "Age": 18,
 "Address": "Dum Dum",
}

print(dict.values())

output:

dict_values(['Arijit', 18, 'Dum Dum'])

(b) fromKeys() → Creates dictionary using this method
Variable name = dict.fromKeys(K, v)
K = Keys, V = Values

(i) You have to store the all keys in a list.

(ii) Then you have to store the values for the given keys.
(iii) It can be created by the help of list.

program → (1)

(i) K = ["Math", "Eng", "Physics", "Chem"]

V = 90

di = dict.fromKeys(K, V)

print(di)

Parameters :

(i) List of Keys [Any Variable]

(ii) Values [Any Variable]

output:

{'Math': 90, 'Eng': 90, 'Physics': 90, 'Chem': 90}

(ii) K = ['a', 'b', 'c']

V = 0

dictionary = dict.fromKeys(K, V)

print(~~K, V~~ (dictionary))

NOTE

Key-Variable = []

Value-Variable =

Variable = dict.fromKeys(K, V)

↑
(dictionary-variable)

output:

{'a': 0, 'b': 0, 'c': 0}

(iii) using none as default value.

Keys = ["a", "b", "c"]

dict = dict.fromKeys(Keys,)

print(dict)

#

When we don't take the values for the key list then the compiler takes the value 'none' automatically.

output:

{'a': None, 'b': None, 'c': None}

(iv) Creating an empty dictionary using fromKeys()

```
Keys = ["a", "b", "c"]  
dict = dict.fromkeys(Keys, " ")  
print(dict)
```

Output:

```
{'a': " ", 'b': " ", 'c': " "}
```

(v)

```
K = [1, 2, 3]
```

```
dict = dict.fromkeys(K, )  
print(dict)
```

Output:

```
{1: None, 2: None, 3: None}
```

(vi)

```
K = [1, 2, 3]
```

```
dict = dict.fromkeys(K, " ")
```

Output:

```
{1: " ", 2: " ", 3: " "}
```

Also we can take the keys and values by passing arguments in the fromkeys() method

• **Syntax** → dict.fromkeys(Keys, Values) → by passing two arguments

```
dict = dict.fromkeys([1, 2, 3], "arjit")
```

Output:

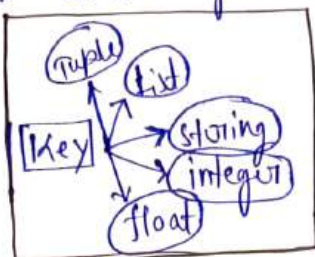
```
{1: "arjit", 2: "arjit", 3: "arjit"}
```

⑥ .get()

.get() method helps us to get the specific values by the key

Syntax:

`dict.get("Key")`



(i)

```
dict = {  
    1: "Arijit",  
    2: "Sourav",  
    3: "Ankit",  
}  
print(dict.get(1))
```

output:

Arijit

```
dict = {  
    1: "Arijit",  
    2: "Sourav",  
    3: "Ankit",  
}  
print(dict.get(2))
```

Output:

Sourav

```
dict = {  
    1: "Arijit",  
    2: "Sourav",  
    3: "Ankit",  
}  
print(dict.get(3))
```

output:

Ankit

if we print a key that is not present at the dictionary then it will occur 'none'.

```
dict = {  
    1: "Arijit",  
    2: "Sourav",  
    3: "Ankit",  
}
```

```
print(dict.get(4))
```

output:

None

| Key-4 is not present at the dictionary,
so it print 'None'.

⑦ .items()

.items() method returns all the elements which is present at dictionary.

Syntax:-

`dict.items()`

①

dict = {

 "name": "Arijit",

 "Age": 19,

 "ID": 231001102245,

 "college": "TUV",

}

print(dict.items())

Output:

dict_items([('Name', 'Arijit'), ('Age', '19'), ('ID', '231001102245'), ('college', 'TUV')])

②

dict = {

 1: 19,

 2: 20,

 3: 21,

 4: 22,

 5: 23,

}

print(dict.items())

Output:

dict_items([(1, '19'), (2, '20'), (3, '21'), (4, '22'), (5, '23')])

⑨ pop()

This method helps to delete the specific key value in the dictionary.

Syntax:

dict.pop()

i>

dict = {

 "name": "Arijit",

 "Age": 18,

 "Id": 2245,

 "college": "TIU",

}

~~print~~(dict.pop("college"))

print(dict)

output:

{'name': 'Arijit', 'Age': 18, 'Id': 2245}

ii>

dict = {

 1: "Arijit",

 2: "Sourav",

 3: "Ankit",

 4: "Sayan",

}

print(dict.pop(4))

print(dict)

output:

{1: 'Arijit', 2: 'Sourav', 3: 'Ankit'}

update()

⑧

Output:

{ 'name': 'Avijit Das', 'Age': 20, 'address': 'Dum Dum Cantt', 'ID': 2245 }

④

dict = {

 "name": "Avijit"

}

dict.update({ "Age": 20 })

print (dict)

Output:

{ 'name': 'Avijit', 'Age': 20 }

Nested Dictionary

→ Nested dictionary in python is a data structure that store another dictionary in a main dictionary.

Syntax:

```
dict = {  
    "Key1": "Value1",  
    "Key2": {  
        "Sub-Key1": "Sub-Value 1",  
        "Sub-Key2": "Sub-Value 2",  
        "Key3": {  
            "Sub-sub-Key1": "Sub-sub-Value 1",  
            "Sub-sub-Key2": "Sub-sub-Value 2",  
        }  
    }  
}  
  
print(dict)
```

① Print your BIO-DATA by using nested dictionary python.

```
dict = {  
    "Name": "Avijit",  
    "BIO-DATA": {  
        "Name": "Avijit Das",  
        "Address": "Dum Dum",  
        "Age": 20,  
        "Father's Name": "Deepankar Das",  
        "Rupali Das": "Mother's Name",  
    }  
}
```

"college": "TIU",
"stream": "BEA",
"SUBJECTS": {

"Sub-1": "BEN",
"Sub-2": "Eng",
"Sub-3": "Hindi",
"Sub-4": "Chemistry",
"Sub-5": "Physics",

"MARKS": {

"Sub-1": 80,
"Sub-2": 100,
"Sub-3": 100,
"Sub-4": 100,
"Sub-5": 100,

}

}

}

}

print(dict)

//Nested Dictionary Methods //

- ① Keys() → Returns only the top level Keys in the dictionary, but does not return the subkeys from the dictionary.

Syntax → `dict.keys()`

Program - 1:

```
dict = {
```

```
    "Key 1" : "Value 1",
```

```
    "Key 2" : "Value 2",
```

```
    "Key 3" : {
```

```
        "Sub-Key1" : "Sub-Value1",
```

```
        "Sub-Key2" : "Sub-Value2",
```

```
    }
```

```
}
```

```
print(dict.keys())
```

Output:

```
dict_keys(['Key1', 'Key2', 'Key3'])
```

- ⊗ It returns only top level ~~two~~ keys.
- ⊗ It ignores the subkeys.

② Values() → It returns the main dictionary values first then it returns the all sub dictionary values of the keys with the keys.

Syntax :- dict.values()

Program - 1 :-

```
dict = {  
    "name": "Arijit Das",  
    "Subjects": {  
        "Phy": 90,  
        "Eng": 90,  
        "Math": 90,  
    }  
}
```

```
print(dict.values())
```

output :-

```
dict_values(['Arijit Das', {'Phy': 90, 'Eng': 90, 'Math': 90}])
```

* It prints the sub keys values with the keys.

"Subjects": { → Sub Key

→ "Phy": 90,
"Eng": 90,
"Math": 90, ← Key + Value

}

output :- {'Phy': 90, 'Eng': 90, 'Math': 90}

② items() → It returns all the key-values which are located at the dictionary and sub dictionaries.

Syntax:- `dict.items()`

Program-1:-

```
dict = {  
    "name": "Arijit Das",  
    "Age": 18,  
    "subjects": {  
        "Phy": 90,  
        "Math": 90,  
        "Beng": 90,  
    }  
}
```

```
print(dict.items())
```

Output:-

```
dict.items([('name', 'Arijit Das'), ('Age', '18'), ('subjects',  
    ['Phy': '90', 'Math': '90', 'Beng': '90'])])
```

It returns all the items. [Key + Values].

We can change any value of a key in a dictionary.

P-1

```
dict = {
```

```
    "name": "Arijit",
```

```
    "Subjects": {
```

```
        "Phy": 90,
```

```
        "Math": 90,
```

```
        "Eng": 100,
```

```
    }
```

```
}
```

~~print~~.

```
dict["Subjects"]["Phy"] = 100
```

```
print(dict)
```

Output:

```
{'name': 'Arijit', 'Subjects': {'Phy': 100, 'Math': 90, 'Eng': 100}}
```

P-2

```
dict = {
```

```
    "name": "Arijit",
```

```
    "Subjects": {
```

```
        "Sub-1": "Beng",
```

```
        "Sub-2": "Eng",
```

```
        "Sub-3": "Math",
```

```
        "Sub-4": "Hindi",
```


② items() → It returns all the key-values which are located at the dictionary and sub dictionaries.

Syntax :- dict.items()

Program-1 :-

```
dict = {  
    "name": "Arijit Das",  
    "Age": 18,  
    "subjects": {  
        "Phy": 90,  
        "Math": 90,  
        "Beng": 90,  
    }  
}
```

```
print(dict.items())
```

output :-

```
dict_items([('name', 'Arijit Das'), ('Age', '18'), ('subjects',  
    ['Phy': '90', 'Math': '90', 'Beng': '90'])])
```

It returns all the items. [Key + Values].

We can specifically print a value by ↓

(P-1)

```
dict = {
```

```
    "Name": "Arijit",
```

```
    "Subjects": {
```

```
        "Sub-1": "Beng",
```

```
        "Sub-2": "Eng",
```

```
        "Sub-3": "Math",
```

```
        "Sub-4": "Chem",
```

```
        "Marks": {
```

```
            "Beng": 90,
```

```
            "Eng": 90,
```

```
            "Math": 90,
```

```
            "Chem": 90,
```

```
        }
```

```
    }
```

```
}
```

```
print(dict["Subjects"]["Marks"])
```

"Marks": {

"Ben": 90,

"Eng": 90,

"Math": 90,

"Hindi": 90,

}

}

}

diet["Subjects"]["Marks"]["Ben"] = 100

print(diet)

Output:

{ 'name': 'Arijit', 'Subjects': { 'Sub-1': 'Beng', 'Sub-2': 'Eng',
'Sub-3': 'Math', 'Sub-4': 'Hindi', 'Marks': { 'Ben': 100,
'Eng': 90, 'Math': 90, 'Hindi': 90 } } }

Output:

90

We can specifically delete a key or subkey in a dictionary.

Syntax:

`del dict[ ]`

```
dict = {
```

```
    "name": "Arjit Das",
```

```
    "Subjects": {
```

```
        "Sub-1": "Math",
```

```
        1: "English",
```

```
        2: "Bengali",
```

```
    "Marks": {
```

```
        "Math": 100,
```

```
        "English": 100,
```

```
        "Bengali": 100,
```

```
    }
```

```
}
```

```
}
```

```
del dict["Subjects"]["Marks"]
```

```
print(dict)
```

output:

```
{'name': 'Arjit Das', 'Subject': {'Sub-1': 'Math', 1: 'English', 2: 'Bengali'}}
```

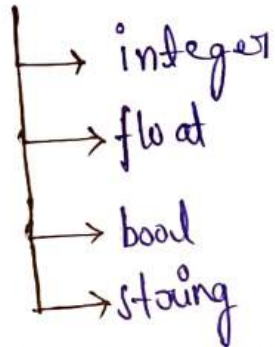

Dictionary Method

keys()	clear()
values()	update()
items()	length()
Normal add	get()
copy()	setdefault()
del	fromkeys()
pop()	loop dictionary
popitem()	

Set data structure

set is a data structure in python which helps to store the data in an immutable way.

- (i) It is immutable.
- (ii) It is unordered.
- (iii) It can store all data types.



- (iv) It can store tuple data structure but not Dictionary and list.

Syntax :-

Set = Set()

Set Methods

add()	loop set
sorted()	union()
remove()	intersection()
discard()	difference()
update()	symmetric_difference()
min()	symmetric_difference_update()
max()	issubset()
pop()	issuperset()
clear()	isdisjoint()

List in Python:

LIST

* A built in data type that stores numbers of values. It can store data types like integer (int), string (str), boolean (bool), float (float).

* A list can store any type of data type in a single list.

```
List = [18, "Amit", 5.0, True]
```

* Basic syntax:

```
List = [int, float, str, bool]
```

```
List = [only integer type values in a single list]
```

```
List = [only float type values in a single list]
```

```
List = [only string type values in a single list]
```

* Integer type values in a list:

```
a = [1, 8, 3, 4, 5] | print(type(a)) | output: <class 'list'>
```

a is a list variable where all the values are integer type.

* float type values in a list:

```
float = [5.0, 18.9, 77.98] | print(type(a)) | output: <class 'list'>
```

We can directly pass name (Variable) as the data type name.

Here float is a list variable where all the values are float type.

* String type values in a list:

```
str = ["Amit", "name", "Address"]
```

Here string is a list variable where all the values are string type.

* `b = [1, 2, 3, 4]`

`print(b)`

output:

`[1, 2, 3, 4]`

* we can specifically print a index value :

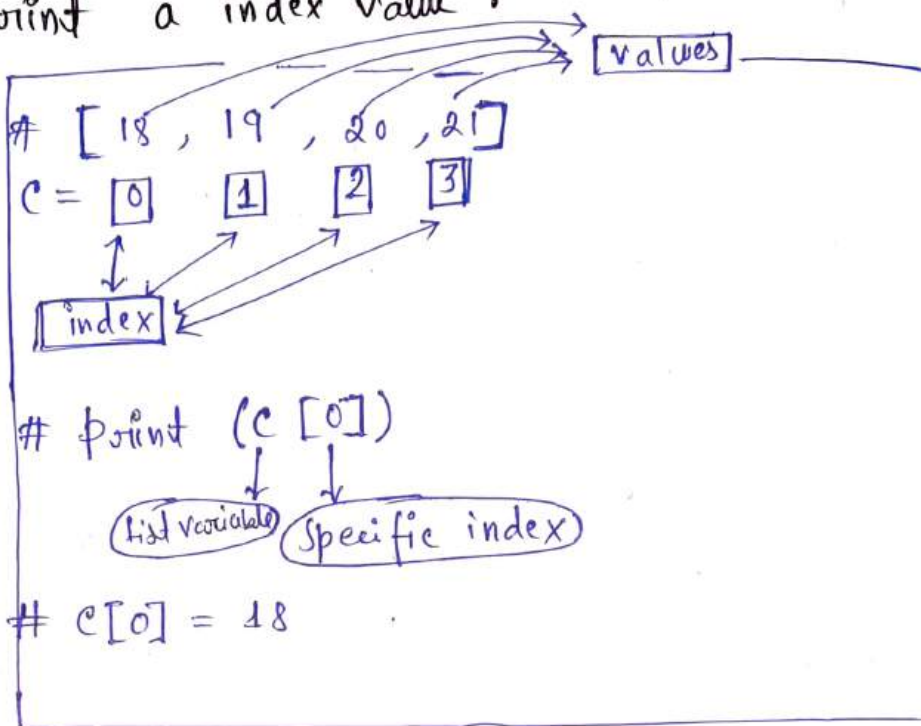
`c = [18, 19, 20, 21]`

`print(c[0])`

output:

18

* we can specifically print the index so that we can access the value of that index.



* List is ~~immutable~~ mutable, so that we can change the index value:

`c = ["Amit", 18, 20.18]`

`c[1] = 20`

`print(c)`

output:

`["Amit", 20, 20.18]`

1st:

`c = ["Amit", 18, 20.18]`
[0] [1] [2]

2nd:-

`c[1] = 20`

Here we particularly mention that index number, so that we can change the value.

So,

We can change the value in a index by mentioning the particularly index, because list is mutable or changeable

* We can print the values which are present in the list.

```
list = [1, 2, 3, 4]
```

```
for list-1 in list:  
    print(list-1)
```

output:

1

2

3

4

* We can print the sum of the integer list values.

```
list = [1, 2, 3, 4, 5]
```

```
a = sum(list)
```

```
print(a)
```

output:

15

```
list = [1, "Arjit"]
```

```
b = sum(list)
```

```
print(b)
```

output:

Error . cannot sum with integer and string data type.

1 = int

"Arjit" = str

* We can print the length of the list.

```
list = [1, "name", "Kolkata", 18.0, True]
```

```
print(len(list))
```

output:

5

Basically it counts the number of values in a list.

The number of values in this list is 5, so the output is 5.

* We can get the maximum and minimum values from the list.

list = [1, 9, 45, 98, 7, 10, 55.56]

print(max(list))

output:

98

list = [1, 9, 45, 98, 7, 10, 55.56]

print(min(list))

output:

1

* find even and odd no. from the list.

list = [1, 4, 3, 19, 20]

for i in list:

if (i % 2 != 0):

print("odd num is:", i)

else:

print("even num is:", i)

output:

~~1 is odd~~ odd num is: 1
~~4 is even~~ even num is: 4
odd num is: 3
odd num is: 19
even num is: 20

using loop

list = [1, 4, 3, 19, 20]

↓
i

i = index which are present in the list.

for i in list:

This for loop syntax is used for printing the indexes from the list.

If the index value ^{give} get the remainder after divided with two then it is even. else it is odd.

List slicing

- (i) Positive Slicing.
- (ii) Negative Slicing.

Syntax:

[starting index : ending index]

(A) Positive Slicing:

- (i) `list = [1, 2, "name", 4.18]`
`print (list[0: length of the list])` / `print (list[0: ])`

output:

1, 2, "name", 4.18

- (ii) `list [1, 2, "name", 4.18]` # print only 1 and 2
`print (list[0:2])`

output:

[1, 2]

- (iii) `list = [1, 2, "name", 4.18]` # print only name and 4.18
`print (list[2: ])`

Output:-

["name", 4.18]

- (iv) `list = [1, 2, "name", 4.18]` # print ~~only~~ all
`print (list[: ])`

output:

[1, 2, "name", 4.18]

(i)

Negative Slicing:Syntax:

[negative 1st index : ~~positive~~ negative last index]

(i)

```
list = [1, 2, 3, 4, 5]
print(list[-5:])
```

print 1 to 4

output:

[1, 2, 3, 4, 5]

(ii)

```
list = [1, 2, 3, 4, 5]
print(list[-3:])
```

print only 3, 4, 5

output:

[3, 4, 5]

(iii)

```
list = [1, 2, 3, 4, 5]
print(list[-5:-2])
```

print only 1, 2, 3

output:

[1, 2, 3]

* Find the average or mean in a list.

$$\text{Avg} = \frac{\sum \text{values in list}}{\text{total values in a list}}$$

① list = [10, 20, 30, 40]

avg = sum(list) / len(list)

print(f"avg: {avg}")

Output:

avg: 25.0

$\sum$ values in list

= total sum of the values in the list.

total values in a list

= We can get the total values by printing the length (len()).

length = Total no. of list.

List Methods

append()	append an element on list
extend()	extend an element on list
copy()	copy all element on list
count()	count an element on list
index()	find an element on list
insert()	insert an element on list
length()	find length an element on list
remove()	remove an element on list
pop()	remove last element on list
reverse()	reverse an element on list
clear()	clear all element on list
sort()	sort all element on list
() + ()	concatenate two list or more
loop list	loop all element on list

Tuples in Python

- * Tuples are used to store multiple items in a single ~~data~~ variable.
- * Tuples are immutable/unchangeable.

Examples

```
this_tuple = ("apple", "banana", "cherry")  
print(this_tuple)
```

- * Tuples are ordered, unchangeable and allow duplicate values.

i) Tuple index:

- * Tuple items are indexed, the first item has index ~~0~~ [0], the second item has index [1]

ii) Allow duplicates:

```
this_tuple = ("apple", "banana", "cherry", "apple", "cherry")
```

iii) Tuple length:

length function determines how many items are present in the tuple.

• eg →

```
tuple = ("apple", "cherry", "banana")  
print(len(tuple))
```

iv) Create tuple with one item:

#1

```
tuple = ("Avigild",)  
print(type(tuple))
```

output:
<class 'str'>

- * (Python compiler will not recognize it as a tuple if you do not add a comma after an item)

#2

```
tuple = ("Avigild",)  
print(type(tuple))
```

output
<class 'tuple'>

v) Data types

A tuple can contain different data types.

tuple 1 = ("abc", 34, True, 40, "male")

v) Creating tuple from another data types.

■ From a list.

```
my-list = [1, 2, 3, 4]
```

```
my-tuple = tuple(my-list)
```

```
print(my-tuple)
```

Output

(1, 2, 3, 4)

■ From a string

```
text = "hello"
```

```
my-tuple = tuple(text)
```

```
print(my-tuple)
```

Output

('h', 'e', 'l', 'l', 'o')

■ From a set

```
my-set = {10, 20, 30}
```

```
tuple-1 = tuple(my-set)
```

```
print(tuple-1)
```

Output

(10, 20, 30)

■ From a dictionary

```
my-dict = {'a': 1, 'b': 2}
```

```
my-tuple = tuple(my-dict)
```

Output

('a', 'b')

⊗ It prints only the keys by default.

■ Printing keys & values both:

```
my_dict = {  
    'a': 1,  
    'b': 2,  
}
```

```
print(my_dict)
```

```
print(tuple(my_dict))
```

```
tuple_dict = tuple(my_dict.items())
```

```
print(tuple_dict)
```

output

```
{'a': 1, 'b': 2}
```

```
('a', 'b')
```

```
(( 'a', 1), ('b', 2))
```

Tuple slicing

```
my_tuple = ("ronaldo", "messi", "neymar", "maradona")
```

```
print(my_tuple[0:2]) # ('ronaldo', 'messi')
```

```
print(my_tuple[: -1]) # All the elements in the tuple
```

```
print(my_tuple[1:3]) # ('messi', 'neymar')
```

Slice by index :

```
("ronaldo", "messi", "neymar", "maradona")
```

[0]
index

[1]
index

[2]
index

[3]
index

Tuple Slicing

i

count()

Returns the numbers of elem times the value appears in the tuple.

```
this_tuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
```

```
x = this_tuple.count(7)
```

```
print(x) # 2
```

ii

index()

Returns the index number where the element is present.

```
this_tuple = (1, 3, 7, 8, 9)
```

```
print(this_tuple.index(3)) # 1
```

NOTE

If the same element in the tuple present many times, It will only print the first index of that element.

```
this_tuple = (1, 3, 7, 8, 9, 8, 10, 9)
```

→ This one will be not count

→ This one will count

```
print(this_tuple.index(9)) # 4
```

```
print(this_tuple.index(8)) # 3
```

Tuple Methods

count()	count element on tuple
index()	find an element on tuple
len()	find length on tuple
min()	find minimum on tuple
max()	find maximum on tuple
sum()	to sum all element on tuple
sort()	to sort all element on tuple
loop tuple	loop all element on tuple

main
methods

functions

Match case statement

Syntax:-

Variable = input from user or specific value.

match Variable:

Case Pattern 1:

print(" ")

Case Pattern 2:

print(" ")

Case _:

print(" ")

Basically it matches the variable taken by the user or a specific value by writing the case patterns.

→ # It takes a by default value.

①

```
num = int(input("Enter a number : "))
```

match num:

Case 1:

print("Number is 1")

Case 2:

print("Number is 2")

Case _:

print("Invalid")

Output:

Enter a number : 2
Number is 2

Q Write a program to print table of a number which lies between 1 and 10.

using for loop

```
num = int(input("Enter a number"))
```

match num:

case 1|2|3|4|5|6|7|8|9|10:

```
for i in range(1, 11):
```

```
    print(i * num)
```

case - :

```
    print("INVALID")
```

output:

Enter a number : 5

5

10

15

20

25

30

35

40

45

50

using while loop

```
num = int(input("Enter a number:"))
```

```
i = 1
```

match num:

case 1|2|3|4|5|6|7|8|9|10:

```
while i <= 10:
```

```
    print(i * num)
```

```
    i += 1
```

Output:

Enter a number: 2

2

4

8

8

10

12

14

16

18

20

② Print Day name of week in Python using match case.

```
day = int(input("Enter day number: "))
```

match day:

case 1:

```
print("Monday")
```

case 2:

```
print("Tuesday")
```

case 3:

```
print("Wednesday")
```

case 4:

```
print("Thursday")
```

case 5:

```
print("Friday")
```

case 6:

```
print("Saturday")
```

case 7:

```
print("Sunday")
```

Output:

Enter day number : 5

Friday

④ Make calculator using match case :

```
num-1 = int(input("Enter first number : "))
```

```
num-2 = int(input("Enter second number : "))
```

```
operator = input("Enter operator (+ - / * % **))
```

match operator :

case '+' :

```
print(num-1 + num-2)
```

case '-' :

```
print(num-1 - num-2)
```

case '/' :

```
print(num-1 / num-2)
```

case '**' :

```
print(num-1 ** num-2)
```

case '%' :

```
print(num-1 % num-2)
```

case - :

```
print("Syntax Error")
```


Loops in Python

■ Defination :-

Loops are used to repeat instructions.

④ There are two types of loops in Py.

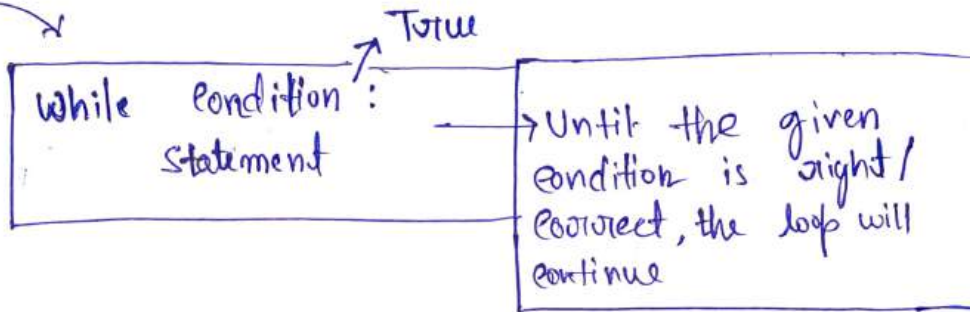
- * → While loop
- for loop

■ While loop

A while loop in Python repeatedly executes a block of code as long as a given condition remains True.

* Syntax :-

While condition :
code to execute



* Codes ↴

① While True:
 print("Hello")

Output :
Hello
Hello
Hello
⋮
∞

* The "hello" will print upto infinite times because the condition is remains True (Always).

* If the condition is false, it will not execute anything.

② a = 10
b = 2

while a*b == 20:
 print("World")

output :
World
World
World
World
World
World
⋮
∞

output

③
a = 10
b = 2
while a/b == 5:
 print("Asirid")

Same as the previous output (2)

output

④
a = 0.1
b = 0.2
c = 0.3
while a+b == c:
 print("Asirid")

Now this code will not run anything because the condition is false.

in language $0.1 + 0.2$ is exactly not 0.3 ,
So it is false.

⑤
list = [1, 4, 5]

list.remove(5)

b = list

while len(b) == 2:
 print("Hello, World")

output

Same as no. 3

output

① Print a name 5 times.

count = 1

while count $\leq$ 5 :

print("Hello, world")

count += 1

Hello, world

Hello, world

Hello, world

Hello, world

Hello, world

② Print 1 to 10 by while loop.

i = 1

while i $\leq$ 10 :

print(i)

i += 1

output

1

2

3

4

5

6

7

8

9

10

③ Print 1 to 100 by while loop.

i = 1

while i $\leq$ 100 :

print(i)

i += 1

output

1

.

.

.

.

.

.

100

④ Print any number in reverse using while loop.

```
num = int(input("Enter any number :"))
```

```
rev = 0
```

```
while num != 0:
```

```
    digit = num % 10
```

```
    rev = rev * 10 + digit
```

```
    num //= 10
```

```
print(rev)
```

Output

Enter any number : 156

651

⑤ Print the ^{Sum of} first n natural number.

```
num = int(input("Enter any number :"))
```

```
i = 1
```

```
Sum-n = 0
```

```
while i <= num:
```

```
    Sum-n += i
```

```
    i += 1
```

```
print(Sum-n)
```


output:

Enter any number: 5

15

⑥ Print the factorial of first n natural numbers.

```
n = int(input("Enter any number: "))
```

```
i = 1
```

```
fact = 1
```

```
while i <= n:
```

```
    fact = fact * i
```

```
    i += 1
```

```
print(fact)
```

output:

Enter any number: 5

120

⑦ Print day name using while loop.

```
day_num = 1
```

```
while day_num <= 7:
```

```
    if day_num == 1:
```

```
        print("Monday")
```

```
    elif day_num == 2:
```

```
        print("Tuesday")
```

```
    elif day_num == 3:
```

```
        print("Wednesday")
```

```
elif day_num == 4:
```

```
    print("Thursday")
```

```
elif day_num == 5:
```

```
    print("Friday")
```

```
elif day_num == 6:
```

```
    print("Saturday")
```

```
elif day_num == 7:
```

```
    print("Sunday")
```

```
if i == 1
```

```
day_num += 1
```

Output:

Monday

Tuesday

Wednesday

Thursday

Friday

Saturday

Sunday

⑦ Another method // using list:

```
days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday",  
        "Sunday"]
```

```
i = 1
```

```
while i <= len(days):  
    print(days[i])  
    i += 1
```

⑧ Print even and odd numbers using while loop. [2 → 20]

```
i = 2
```

```
Print("Even numbers: ")
```

```
while i <= 20:  
    if i % 2 == 0:  
        print(i)  
    i += 1
```

```
Print("Odd numbers: ")
```

```
i = 2
```

```
while i <= 20:  
    if i % 2 != 0:  
        print(i)  
    i += 1
```

Output

Even numbers:

2

4

6

8

10

12

14

16

18

20

Odd numbers:

3

5

7

9

11

13

15

17

19

20

11) Count the number of digits in a given number using while loop.

```
num = int(input("Enter number : "))
```

```
count = 0
```

```
if num == 0:
```

```
    count = 1
```

```
else:
```

```
    while num > 0:
```

```
        num //= 10
```

```
        count += 1
```

```
print(count)
```

Output:

```
Enter number : 123
```

```
3
```

12) Check the number is palindrome or not.

```
num = int(input("Enter any number : "))
```

```
orig_num = num
```

```
rev = 0
```

```
while num > 0:
```

```
    dig = num % 10
```

```
    rev = rev * 10 + dig
```

```
    num //= 10
```

```
if orig_num == rev:
```

```
    print("Number is Palindrome")
```

```
else:
```

```
    print("Not a Palindrome")
```


Output:

Enter a number: 144

number is Palindrome

19

check the number is Armstrong or not. Digit

$n = \text{int}(\text{input}(\text{"Enter a number:"}))$

Sum = 0

org_n = n

order = len(str(n)) # converts the int to str and calculate the length of str

while n > 0:

dig = n % 10 # get the remainder of the number (n) and separate

Sum = Sum + dig ** order

0 + dig ** order on length of the number (str)

n //= 10 # divide until the num become zero

if org_n == Sum:

print("Palindrome")

else:

print("Not Palindrome")

output:

Enter a number: 153

Palindrome



$$1^3 + 1^3 + 1^3 = \boxed{111}$$

Armstrong = 111 and sum

→ If the cube of digits in a number is equal to the number is called Armstrong number.

1 2 3

$$1^3 + 2^3 + 3^3 = 1 + 8 + 27 = 36$$

not Armstrong number

Dry Run code

153

153 % 10 → 3
15 % 10 → 5
1 % 10 → 1

→ first step
→ Digits

2nd step → order(len) ** digits

$$\text{Sum} = \text{Sum} + \text{dig} ** \text{order} \\ = 0 + 3 ** 3 = 27$$

$$\text{Sum} = \text{Sum} + \text{dig} ** \text{order} \\ = 27 + 5 ** 3 = 152$$

$$\text{Sum} = \text{Sum} + \text{dig} ** \text{order} \\ = 152 + 1 ** 3 = 153$$

3rd step → ~~num~~ n // = 10

$$153 // 10 \rightarrow 15$$

$$15 // 10 \rightarrow 1$$

$$1 // 10 \rightarrow 0$$

14) GCD of two numbers using while loop.

```
a = int(input("Enter first number: "))
```

```
b = int(input("Enter second number: "))
```

```
i = 1
```

```
while a % i == 0 and b % i == 0:
```

```
    gcd = i
```

```
    i = i + 1
```

```
print(gcd)
```

output:-

Enter first number: 12

Enter second number: 6

3

15) Sum of all even digits in a number using while loop.

```
n = int(input("Enter a number: "))
```

```
sum = 0
```

```
while n > 0:
```

```
    dig = n % 10
```

```
    if dig % 2 == 0:
```

```
        sum = sum + dig
```

```
    n //= 10
```

```
print(sum)
```

output

Enter a number: 4578

12

FUNCTIONS :

Definition

A function in Python is a block of code that performs a specific task. You can use it by calling its name only. It saves you from writing the same code repeatedly.

Basic Concept :

- create: creates the function by writing `def` keyword to define function.
- call: We have to call the function after define the function.
- Parameter: We have to pass the parameter in the `define(def)` keyword.
- Arguments: It depends on the parameter. It pass the actual parameter value when we call it.

Types of function

Built-in library function

user-defined function

① Built-in library function :-

`Print()`, `Sum()`, `max()`, `min()`, `input()`, `type()`.

② user-defined function :-

`def func-name (Para1, Para2):` → define function in function name

`# block of code` → function name

`return value` → block of code / value

→ returns the value

`func-name()`

→ calling the function

Basics

USER-DEFINE FUNCTION

- Without Passing Parameter
- Passing Parameter

① Without Passing Parameter:

① greetings ()

```
def greetings():  
    print("Hello world")
```

```
greetings()
```

output:

Hello world

② Passing Parameter:

① Summation #

```
def Sum(a, b):
```

```
    add = a + b
```

```
    print("Addition: ", add)
```

```
Sum(1, 2)
```

output:

Addition: 3

def Sum(a, b):

```
    add = a + b
```

```
    return add
```

```
print(Sum(2, 3))
```

Output:

5

(ii)

Power of a number:

num = int(input("Enter the power of a number:"))

def power(a):

pwr = a**num

return pwr

print(power(5))

output

Enter the power of a number: 2

25

(2) Polymorphism in function:

* What is Polymorphism in function or in Python??

⇒ Polymorphism means repetition of a string that multiplies with an integer.

eg:→

print(5*'h')

output:

hhhhh

print(3*'h\#')

output:

h h h

Basically polymorphism means string multiplied by integer.

In function

```
def pol(a, b):
```

```
    return a * b
```

```
print(4, 'h')
```

output

h h h h

Taking user input:

```
a = int(input("Enter Integer :"))
```

```
b = str(input("Enter string :"))
```

```
def pol(a, b):
```

```
    plm = a * b
```

```
    print("\n polymorphism : ", plm)
```

```
pol(a, b)
```

Output:

Enter Integer : 3

Enter string : t

polymorphism : t t t

Function returning multiple values.

```
def add # Default input  
def calculate(a, b):  
    add = a + b  
    min = a - b  
    div = a // b  
    rem = a % b  
    mul = a * b  
  
    return add, min, div, rem, mul
```

Output

8
2
1
2
15

```
print(calculate(5, 3))
```

Taking input from user.

```
a = int(input("Enter first number: "))
```

```
b = int(input("Enter Second number: "))
```

```
def calculate(a, b):
```

```
    add = a + b
```

```
    sub = a - b
```

```
    mul = a * b
```

```
    div = a // b
```

```
    rem = a % b
```

```
    print(add, sub, mul, div, rem)
```

```
calculate(a, b)
```

Output

Enter first number: 5

Enter second number: 3

8

2

15

2

1

Examples on Functions ||

① WAP to print factorial of a number using functions.

```
def factorial(n):  
    fact = 1  
    i = 1  
    while i <= n:  
        fact *= i  
        i += 1  
    return fact  
print(factorial(5))
```

Output

120

② WAP to print Sum of digits using functions.

```
def sum-of-dig(n):  
    sum-n = 0  
    while n != 0:  
        rem = n % 10  
        sum-n += rem  
        n //= 10  
    return sum-n  
print(sum-of-dig(124))
```

Output

7

③ WAP to print only even and odd numbers and separate them in a list using function.

```
def even-odd-num(n):  
    list-1 = []  
    list-2 = []  
    for n in numbers:  
        if n % 2 == 0:  
            list-1.append(n)  
        else:  
            list-2.append(n)  
    return list-1, list-2
```

```
numbers = [1, 2, 3, 4, 5, 6]  
even, odd = even-odd-num(numbers)  
print(even, odd)
```

Output

[2, 4, 6] [1, 3, 5, 7]

Important Python Functions

Input/Output Functions

- `print()`: Display output
- `input()`: Get user input

Math Functions

- `abs()`: Absolute value
- `pow()`: Power calculation
- `round()`: Round number
- `min()`: Smallest value
- `max()`: Largest value
- `sum()`: Sum of elements

File Handling Functions

- `open()`: Open file
- `close()`: Close file
- `read()`: Read file
- `write()`: Write file

Miscellaneous Functions

- `help()`: Get documentation
- `dir()`: List attributes
- `globals()`: Get global variables
- `locals()`: Get local variables
- `callable()`: Check if callable

Memory Management Functions

- `id()`: Get memory address
- `hash()`: Hash value

Iterators & Generators

- `iter()`: Get iterator
- `next()`: Next element
- `range()`: Generate sequence

Type Conversion Functions

- `int()`: Convert to integer
- `float()`: Convert to float
- `str()`: Convert to string
- `bool()`: Convert to boolean
- `list()`: Convert to list
- `tuple()`: Convert to tuple
- `set()`: Convert to set
- `dict()`: Convert to dictionary

String Functions

- `ord()`: Unicode of character
- `chr()`: Character from Unicode
- `format()`: Format string
- `repr()`: String representation

Sequence Functions

- `len()`: Length of object
- `sorted()`: Sort items
- `reversed()`: Reverse items
- `enumerate()`: Index with elements
- `zip()`: Pair elements

Functional Programming Func

- `map()`: Apply function
- `filter()`: Filter elements
- `reduce()`: Apply cumulative function
- `lambda`: Anonymous function

Error Handling Functions

- `eval()`: Evaluate expression
- `exec()`: Execute code
- `compile()`: Compile code



OOPS

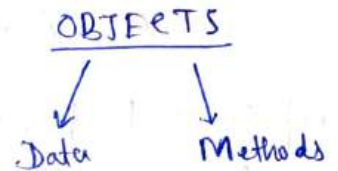
Object-Oriented Programming

OOPs

Object oriented Programming

■ OBJECT

- Objects are Real world entities.
- Entities have their own data and methods.



Example:

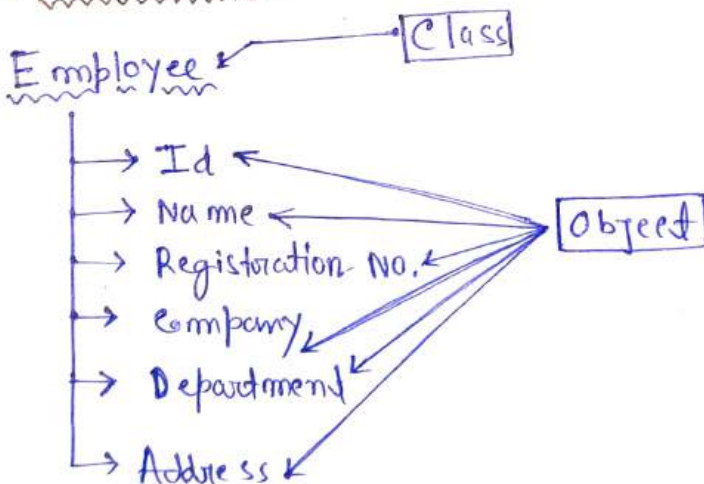
- An object or an Entity or an real world thing.
- A Watch or clock which have it's own data and methods
- A watch has it's own data like, it's colour, brand, size, materials.
- Methods means how the watch is working, basically the function behind it.

■ class

- classes is a group of objects. or real world entities.
- classes are mainly a "blueprint" for creating a object.

⊛ Basically Object is depended on class.

■ EXAMPLE:



▲ NOTE

- Basically Employee is a class and the details about the employee is objects.

• NOUN: Employee [class]

• Adjective: Details about the Employee [object]

Syntax

(i) Class Employee:
 name = "Arijit"

output
Arijit

Emp_obj = Employee()
~~print(Employee.name)~~
print(Emp_obj.name)

(ii) class Employee:
 name = "Arijit"
 age = 18

output
Arijit
18

Emp_obj = Employee()
print(Emp_obj.name, Emp_obj.age)

Basic syntax

class Employee:
 def __init__(self, name, age):
 self.name = name
 self.age = age

 def display(self):
 print("Name is :", self.name)
 print("Age is :", self.age)

Emp = Employee("Arijit", 20)
Emp.display()

output

Name is : Arijit
Age is : 20

1. Class Definition

```
class Employee:
```

2. Constructor method --init--

```
def __init__(self, name, age):
```

```
    self.name = name  
    self.age = age
```

Attributes of the object

- creates the class

- Basically it is the blueprint of the class

- `--init--` is a special method in Python known as the constructor.

- It is automatically called when a new object of the class is created.

- `self` refers to the current object.

- `name`, `age` are the parameters passed during object creation.

3. Display Method

```
def display(self):
```

```
    print("Name is:", self.name)  
    print("Age is:", self.age)
```

User-defined function

- `display` is a user-defined function.

- used to print the attributes of the object

4. Creating an object

```
Emp = Employee("Arifid", 20)
```

```
Emp.display
```

- Here we created the object of the class `Employee`

- Here the `--init--` method called when the object is created

- `"Arifid"` is passed to `name` and `20` is passed to `age`.

Constructor

① --init--():

⇒ --init--() is a constructor method which automatically initialize the objects or the instances of the class

⇒ It initialize newly created objects.

① Syntax

```
class ClassName:
```

```
    def __init__(self, parameters):
```

② Types of Constructor:

① Default Constructor:

```
class Example:
```

```
    def __init__(self):
```

```
        print("Default Constructor")
```

```
obj1 = Example()
```

⇒ That constructor has no parameter except self.

② Parameterized Constructor:

```
class Car:
```

```
    def __init__(self, model_name):
```

```
        self.model = model_name
```

```
obj1 = Car("BMW")
```

```
print(obj1.model)
```

⇒ That constructor has many parameter.

ii) class Student:
 def __init__(self, s_name, s_age, s_id):
 self.name = s_name
 self.age = s_age
 self.id = s_id

Arijit	20	2245
Rahul	20	2246
Ankit	20	2247

S1 = Student("Arijit", 20, 2245) → object 1

print(S1.name, S1.age, S1.id)

S2 = Student("Rahul", 20, 2246) → object 2

print(S2.name, S2.age, S2.id)

S3 = Student("Ankit", 20, 2247) → object 3

print(S2.name, S2.age, S2.id)

ⓧ --init-- Constructor automatically initialize those objects when it is created one by one.

ii) Self keyword refers to that instances of the class [objects].

iii) WAP to print the name of student and three subs and the average of the three sub marks. Method example OOPS Methods

class Student:

def __init__(self, name, sub1, sub2, sub3, sub1_m, sub2_m, sub3_m):

 self.name = name

 self.sub1 = sub1

 self.sub2 = sub2

 self.sub3 = sub3

 self.sub1_m = sub1_m

 self.sub2_m = sub2_m

 self.sub3_m = sub3_m

def show_marks(self):

 print("Student name is :", self.name)

 print("First Subject is :", self.sub1)

 print("Second Subject is :", self.sub2)


```
print("Third Subject is:", self.sub3)
print("First Subject marks:", self.sub1-m)
print("Second Subject marks:", self.sub2-m)
print("Third Subject marks:", self.sub3-m)
```

```
s1 = student("Arigid Das", "Bengali", "English", "Math", 90, 92, 93)
s1.show_marks()
```

Methods

Methods are functions that belong to objects.

Creating class

```
class Name:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def show(self):
```

```
        print("Your name is:", self.name)
```

```
n1 = Name("Arigid")
n1.show()
```

Output

Your name is: Arigid

① WAP to print name & three sub marks average.

class Student:

```
def __init__(self, name, marks):
```

```
    self.name = name
```

```
    self.marks = marks
```

```
def show-average(self):
```

```
    for i in self.marks:
```

```
        sum-n += i
```

```
    print(self.name, "your average is : ", sum-n/3)
```

```
s1 = Student("Asirid", [10, 20, 30])
```

```
s1.show-average()
```

Output

Asirid, your average is : 20.0

Private (like) attributes & methods

Private attributes & methods are meant to be used only within the class and are not accessible from outside the class.

self. -- attribute

■ example:

```
class Account:
```

```
    def __init__(self, ac-no, ac-pass):
```

```
        self.ac-no = ac-no
```

```
        self.__ac-pass = ac-pass
```

```
a1 = Account('ABCD', '1234')
```

```
print(a1.ac-no) # This will print.
```

```
print(a1.__ac-pass) # This will not print because it is private.
```

■ But we can access that private attribute by another method.

```
class Account:
```

```
    def __init__(self, acc-no, acc-pass):
```

```
        self.acc-no = acc-no
```

```
        self.__acc-pass = acc-pass
```

```
    def reset-pass(self)
```

```
        print(self.__acc-pass)
```

OOP - Inheritance

Defn

Inheritance is an object-oriented programming concept where a child class can acquire properties and methods from a parent class. (superclass)

Example

```
class Car:
```

```
    @staticmethod
```

```
    def start():
```

```
        print(f"Car started.....")
```

```
    @staticmethod
```

```
    def stop():
```

```
        print(f"Car stopped!")
```

```
class ToyotaCar(Car):
```

```
    def __init__(self, model, colour):
```

```
        self.model = model
```

```
        self.colour = colour
```

```
    def show(self):
```

```
        print(f"Model :", self.model)
```

```
        print(f"Colour :", self.colour)
```

```
car1 = ToyotaCar('BMW', 'Blue')
```

```
car1.start() # This will print the parent class method output.
```

```
car1.stop() # Same as car1.start()
```

```
car1.show() # This will show child class method output.
```

```
a1 = Account('ABED', '1234')
```

```
print(a1.ace-no)
```

a1. reset-pass # This will now print the private pass.

Also we can create private method also.

```
class Account:
```

```
    def __init__(self, ace-no, ace-pass):
```

```
        self.ace-no = ace-no
```

```
        self.ace-pass = ace-pass
```

```
    def __show(self)
```

```
        print(self.ace-no)
```

```
        print(self self.ace-pass)
```

```
a1 = Account('1234', 'abcd')
```

a1.__show() # that will not work

⇒ Notes

→ When Python sees __show, it applies name mangling

→ Initially, your method is stored as _Account__show.

→ So, from outside, a1._show() is not found.

→ You will get attribute error

Example 2

```
class Account:
```

```
    def __init__(self, acc_hold, acc_num):
```

```
        self.acc_hold = acc_hold
```

```
        self.acc_num = acc_num
```

```
    def show(self):
```

```
        print(f"Account holder name: ", self.acc_hold)
```

```
        print(f"Account number: ", self.acc_num)
```

```
class details(Account):
```

```
    def show_again(self):
```

```
        self.show()
```

```
acc1 = details('Anirudh Das', '1234')
```

```
acc1.show_again()
```

Output

```
Account holder name: Anirudh Das
```

```
Account number: 1234
```

Super() Keyword

- Super() keyword is used to call methods from parent class.
- Mainly it is used in a child class.

Types of Inheritance

→ Single-Level Inheritance

→ Multilevel Inheritance

→ Multiple Inheritance

Single-Level Inheritance

Defn:

one parent class and one child class (Just one step of inheritance).

Example:

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def show(self):
```

```
        print(f"Your name is : ", self.name)
```

```
        print(f"Your age is : ", self.age)
```

```
class movieDetails(Person):
```

```
    def __init__(self, name, age, address):
```

```
        super().__init__(name, age)
```

```
        self.address = address
```

```
def show_again(self):
```

```
    print(f"Your balance is : ", self.address)
```

```
P1 = more_details ('Artist Das', 20, 'Dum Dum')
```

```
P1.show() # Parent class
```

```
P1.show_again() # child class
```

Visual Representation

PARENT class Parent



CHILD child class

```
class Parent:
```

```
    pass
```

```
class Child:
```

```
    pass
```

→ one Parent

→ one child

Multi-level-Inheritance

Defn

When a child class inherits from a parent class, and another class inherits from that child class, that is called Multi-level-Inheritance.

Example:

```
class Car:
```

```
    def __init__(self, model):
```

```
        self.model = model
```

```
    def show(self):
```

```
        print(f"model name is :", self.model)
```

```
class details(Car):
```

```
    def __init__(self, model, colour):
```

```
        super().__init__(model)
```

```
        self.colour = colour
```

```
    def show-again(self):
```

```
        print(f"colour is :", self.colour)
```

```
class details-2(details):
```

```
    def __init__(self, model, colour, type):
```

```
        super().__init__(model, colour)
```

```
        self.type = type
```



```
def show-again-2(self):
```

```
    print(f" type is :", self.type)
```

```
Car 1 = details-2 ('BMW', 'Blue', 'Petrol')
```

```
Car 1. show()
```

```
Car 1. show-again()
```

```
Car 1. show-again-2()
```

Visual Representation

Grand Parent



Parent



child

```
class Grand Parent:
```

```
    pass
```

```
class Parent(Grand Parent):
```

```
    pass
```

```
class child(Parent)
```

```
    pass
```

Multiple-Inheritance

defn: Multiple inheritance is a feature in python where a class can inherit attributes and methods from more than one parent class.

It allows you to access the properties (variables) and behaviours (methods) of multiple base classes.

example:

```
class Father:
```

```
    def skill(self):
```

```
        print(f"Father's skill: Gardening")
```

```
class Mother:
```

```
    def talent(self):
```

```
        print(f"Mother's skill: Singing")
```

```
class Child(Father, Mother):
```

```
    def hobby(self):
```

```
        print(f"Child's hobby: Gardening, Singing, Painting")
```

```
c = Child()
```

```
c.skill() # inherit from father
```

```
c.talent() # inherit from mother
```

```
c.hobby() # Defined in child
```

MRO (Method Resolution Order)

Def'n

MRO (Method Resolution Order) is the order in Python which searches classes to find a method or attribute when multiple inheritance is used.

Simple words:

If a class inherits from multiple classes, **MRO** decides which parent class method is called first.

Why MRO needed?

Because in multiple inheritance, two or more parent classes may have methods with the same name.

⊛ Python needs a clear rule to avoid confusion.

Syntax:

`print (child class.mro())`

Code

Output

`[<class '__main__.e'>, <class '__main__.A'>, <class '__main__.B'>]`

```
class A:
```

```
    def show (self):  
        print (f" A ")
```

```
class B:
```

```
    def show (self):  
        print (f" B ")
```

```
class e (A, B):
```

```
    def show (self):  
        print (' A , B , e ')
```

```
print (e.mro())
```


Abstraction

Defn: Abstraction means to show the essentials and hides the internal functionalities.

Example: ① When you drive a car, you just use the brake, accelerator, steering, but you don't worry about how the engine works internally.

⊛ Hides internal works.

⊛ Shows essentials.

Code:

```
class Car:
```

```
    def __init__(self):
```

```
        self.acc = acc
```

```
        self.brake = brake
```

```
        self.clutch = clutch
```

This is the engine of the car, that is hidden

```
    def start(self):
```

```
        print("Accelerator:", self.acc)
```

```
        print("Brake:", self.brake)
```

```
        print("Clutch:", self.clutch)
```

You only show this

```
car1 = Car()
```

```
car1.start()
```


Encapsulation

Defn:

Encapsulation is all about hiding or controlling access to class attributes and methods. Decides what to hide, what to access.

Code:

Concepts: Public, Private, Protected

Class Bank:

```
def __init__(self, bank-name, acc_num, acc_pass):
```

```
    self.bank-name = bank-name
```

```
    self._acc_num = acc_num
```

```
    self.__acc_pass = acc_pass
```

```
def show(self):
```

```
    return self.bank-name, self._acc_num
```

```
b = Bank('SBI', '432102', 123)
```

```
print(b.__acc_pass) # Error
```

```
print(b._acc_num) # Executed
```

- * You can not directly access private attributes by their name.
- * If you want to access it, you have to access it through a method.
- * Public $\Rightarrow$ self.bank-name
- * Private $\Rightarrow$ self._acc-pass ['--] $\leftarrow$ Double underscore
- * Protected $\Rightarrow$ self.__acc-num ['-'] $\leftarrow$ Single underscore

Polymorphism

■ What is Polymorphism?

⇒ Polymorphism means one name many forms.

- Poly → Many
- Morph → Forms

In OOP in python ^{Same} an method / function behaves differently depending on the class / object.

This is called duck-Typing also



■ Real life example:

Think of the word 'run':

↳ A man runs

↳ A dog runs

↳ A cat runs



- Here "runs" is the method / function that behaves differently depending on the place / class / object.

```
class Man:
    def run(self):
        print("A man runs!")

class Dog:
    def run(self):
        print("A Dog runs!")

class Cat:
    def run(self):
        print("A cat runs!")
```

■ Why we use it?

⇒ When different classes performs the same quality.

⇒ When you want to achieve run time flexibility.

■ Parts of Polymorphism ↓

→ Compile - Time Polymorphism
(Method overloading)

→ Run-Time Polymorphism
(Method overriding)

→ Operator Overloading

Compile - Time Polymorphism : (Method overloading)

* Same function name, different arguments

* There are two types of arguments present.

- Default arguments
- Variable arguments

① DEFAULT ARGUMENTS :

class Calculator:

def add (self, a=0, b=0, c=0):

return a+b+c

cal = Calculator()

print (cal.add (1,2)) # 3

print (cal.add (1,2,3)) # 6

② VARIABLE ARGUMENTS (*Args) :
→ You can take multiple arguments on compile time.

Class Calculator:

```
def mul(self, *nums):  
    result = 1  
    for num in nums:  
        result *= num  
  
    return result
```

Cal = Calculator()

print (Cal.mul(2,3,4)) # 24

Run-Time Polymorphism (Method Overriding)

Defn

Method overriding is an OOP feature where a child class inherits quality from Parent class.

- The classes names are different.

```
└→ class Parent  
    └→ class child1  
    └→ class child2
```

- The quality or methods or functions are same.

```
└→ Parent (quality = work)  
    └→ child1 (work)  
    └→ child2 (work)
```


Example

```
class Parent:
```

```
    def work(self):
```

```
        print(f"Parent works hard!")
```

```
class Child1(Parent):
```

```
    def work(self):
```

```
        print(f"Child1 works hard also!")
```

```
class Child2(Parent):
```

```
    def work(self):
```

```
        print(f"Child2 works hard also!")
```

```
classes = [Parent(), Child1(), Child2()]
```

```
for person in classes:
```

```
    person.work()
```

⑧ VARIABLE ARGUMENTS (*Args):

→ You can take multiple arguments on compile time.

Class Calculator:

```
def mul(self, *nums):
```

```
    result = 1
```

```
    for num in nums:
```

```
        result *= num
```

```
    return result
```

```
cal = Calculator()
```

```
print(cal.mul(2,3,4)) # 24
```

Run-Time Polymorphism (Method Overriding):

Defn

Method overriding is an OOP feature where a child class inherits quality from Parent class.

- The classes names are different.

↳ class Parent

↳ class child1

↳ class child2

- The quality or methods or functions are same.

↳ Parent (quality = work)

↳ child1 (work)

↳ child2 (work)

OPERATOR OVERLOADING

```
class Quantity:
    def __add__(self, other):
        new_value = self.value + other.value
        return Quantity(new_value)

    def __sub__(self, other):
        new_value = self.value - other.value
        return Quantity(new_value)

    def __sub__(sub, other):
        new_value = self.value - other.value
        return Quantity(new_value)

    def __mul__(self, other):
        new_value = self.value ** other.value
        return Quantity(new_value)

    def __truediv__(self, other):
        new_value = self.value / other.value
        return Quantity(new_value)

    def __truediv__(self, other):
        new_value = self.value // other.value
        return Quantity(new_value)

    def __mod__(self, other):
        new_value = self.value % other.value
        return Quantity(new_value)

    def __str__(self):
        return 'Quantity[' + str(self.value) + ']'
```

A) Exception Handling:

An exception is an error that occurs during the execution of a program. When Python encounters an error, it stops the execution & shows an error message.

eg →

```
print(10/0)    # Zero Division Error  
int("abc")    # ValueError
```

◆ Why we use Exception Handling?

To avoid crashing the program and instead handle the error gracefully using try and except blocks.

eg →

```
try:  
    a = 10  
    b = 0  
    print(a/b)  
except:  
    print("You can not divide by 0!")
```

output

You can not divide by 0!

i) Types of errors:

- Syntax Error
- Runtime Error
- Logical Error
- Type Error
- Name Error
- Value Error
- Index Error
- Key Error

i) Syntax Error:

```
print(1*2
```

SyntaxError: '(' was not closed.

◆ By using try, except block:

```
try:
```

```
    eval("print(1*2)")
```

```
except:
```

```
    print("Syntax Error!")
```

Syntax Error

ii) Runtime Error: Errors occurs during program execution.

```
print(10/0) # ZeroDivisionError: division by zero
```

◆ By using try, except blocks:

```
try:
```

```
    print(10/0)
```

```
except:
```

```
    print("RunTime Error")
```

iii

Logical Errors: Program runs without crashing, but gives wrong input.

```
def square(x):
    return x*2
```

```
square(5) # 10
```

* This cannot be treated as error because the program executes properly.

* Logical Errors are not exceptions.

iv

Type Error: Performing an operation on the wrong ^{data} type.

```
a = "10"
```

```
b = 5
```

```
print(a+b) # TypeError: can only concatenate str (not "int") to str
```

◆ By using try, except block:

```
try:
```

```
    a = "10"
```

```
    b = 5
```

```
    print(a+b)
```

```
except:
```

```
    print("Type Error")
```

```
# TypeError
```

(v)

Name Error: A variable that is not defined.

```
print(x) # name 'x' is not defined
```

◆ By using try, except blocks:

```
try:
```

```
    print(x)
```

```
except:
```

```
    print("Name Error")
```

```
# Name Error
```

(vi)

Value Error: Function receives the right type but inappropriate value.

```
int("abe") # ValueError: invalid literal for int() with base 10: 'abe'
```

◆ By using try, except blocks:

```
try:
```

```
    int("abe")
```

```
except:
```

```
    print("Value Error")
```

```
# Value Error
```

(vii) Index Error: Accessing an invalid index in a list.

```
list_1 = [1, 2, 3, 4]
```

```
print(list_1[5]) # Index Error: list index out of range
```

◆ By using try, except blocks:

```
try:
```

```
list_1 = [1, 2, 3, 4]
```

```
print(list_1[5])
```

```
except:
```

```
print("Index Error")
```

```
# Index Error
```

(viii)

Key Error:

```
student = {
```

```
    "name": "Anirid",
```

```
    "age": 20,
```

```
}
```

```
print(student["grade"]) # Key Error: 'grade'
```

◆ By using try, except blocks:

```
try:
```

```
student = {
```

```
    "name": "Anirid",
```

```
    "age": 20,
```

```
}  
print(student["grade"])
```


except:

```
print("Key Error")
```

Key Error

(ix)

Import Error: An invalid module that is not exist that you try to import.

```
import maths
```

```
print(maths.sqrt(2))
```

Module Not Found Error: No module named 'maths'

◆ By using try, except blocks:

try:

```
import maths
```

```
print(maths.sqrt(2))
```

except:

```
print("Import Error")
```

Import Error

File handling || FILE I/O ||

To perform any operation (read/write) on a file, you first need to open the file by using `open()` function.

Syntax:

`file_object = open('filename', 'mode')`

- **file name**: Name of the file (can be relative or absolute path).
- **'mode'**: Mode in which the file is opened (r, w, a, etc.).

MODES

- 'r' → Open for reading (By default).
- 'w' → Open for writing, truncating the file first.
- 'x' → Create a new file and open it for writing.
- 'a' → Open for writing, appending to the end of the file.
- 'b' → binary mode.
- 't' → text mode.
- 't' → open a disk file for updating (reading & writing).

Read from a file

Once file open, you can read from a file using these methods.

- `read()`
- `readline()`
- `readlines()`

* At first you have to create a file in your code editor then you can access it by writing this code.

#code

```
file_object = open("file_name.txt", "r")
```

```
Content = file_object.read()
```

```
print(Content)
```

```
file_object.close()
```

File.txt ← A file saved in your editor.

Hello World !!!
I am pursuing AI/ML.

 } → This is file name file.txt

* How can i access it ???

FileHandling.py ← Python file

```
file_object = open("file.txt", "r")
```

```
Content = file_object.read()
```

```
print(Content)
```

```
file_object.close()
```

Output

Hello World !!!

I am pursuing AI/ML.

* Other side you can access it by writing this syntax. Here you cannot have to write that `file.close()`. Here it's automatically closed.

~~code~~ Code

```
with open("file.txt", "r") as f:
```

```
    content = f.read()
```

```
    print(content)
```

Write a file

'w' mode overwrite the file data if the file already exist. If it is not exist then it automatically creates the file.

* Accessing the same file $\Rightarrow$ file.txt

```
with open("file.txt", "w") as f:
```

```
    content = f.write("Arijit!")
```

```
    print(content)
```

Output

Arijit

At first when we created the file as file.txt, there was only written
Hell o world !!!
I am pursuing AI/ML.

But now it overwrite the file. because it's exist already.

If a file does not exist, it automatically creates and inserts the data into it.

With `open("File_2.txt", "w")` as `f`:

```
content = f.write("Hello world again!!!")
```

```
print(content)
```

Output

20

It will give 20 as output because in that File_2.txt file, there are 20 characters present.

Hello World again !!!

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----

 ← Total 20 character present.

Appending data

If you want to append new data into an existing file.

```
f = open('file.txt', 'a')
```

```
data = f.write('new data')
```

```
print(data)
```

```
f.close()
```

→ `lambda()`→ `map()`→ `filter()`→ `reduce()`

1 lambda function:

Usage: • We need lambda function because sometimes we are going to define a small function for a specific task.

• Instead of defining that function again & again we can use `lambda()`.

• It is a built-in function.

2 **Example:** defining function

```
def squares(x):
    return x**2

print(squares(2)) # 4
```

using lambda function

```
squares = lambda x: x**2
print(squares(2)) # 4
```

3 map function:

Usage: • It applies to each element of an iterable.

- Like → [List, tuple, etc...]
- Returns list or tuple always.

Example:

```
nums = [1, 2, 3, 6, 7]
```

```
Squares = list(map(lambda x: x**2, nums))
```

```
print(Squares)
```

```
# [1, 4, 9, 36, 49]
```

③ Reduce function:

Usage:

- It reduces an iterable to cumulative value.
- It always returns a single element.
- It applies function pairwise.
- You have to import functools module, because it belongs to that module.

Example: import functools

```
nums = [1, 2, 3, 4]
```

```
cumsum = reduce(lambda x, y: x+y, nums)
```

```
print(cumsum)
```

```
# 10
```

④ filter function:

- Usage:
- It filters elements from iterable based on True/False
 - Returns only those elements which satisfies condition

Example:

```
nums = [10, 20, 30, 35]
```

```
evens = list(filter(lambda x: x%2==0, nums))  
print(evens)
```

```
# [True, True, True, False]
```

Short Summary

→ Syntax:

→ map():

```
map(function, iterable)
```

→ reduce():

```
reduce(function, iterable)
```

→ filter():

```
filter(function, iterable)
```